

VeriCase — Intro & Architecture (vNext)

This document is the source of truth for the MVP architecture. It reflects the current codebase (backend, frontend, infra) and the tech spec in `docs/TECH_SPEC.md`.

Executive Summary

- Goal: Make it fast to assemble, search, and argue a case.
- Core: Evidence ingestion, hybrid semantic search, claims + rebuttal workflows, and exports.
- Stack: FastAPI (Python), React + TypeScript (Vite), PostgreSQL, OpenSearch (BM25 + k-NN), OpenAI (server-side).
- Guarantee: AI and keys live server-side; browser holds no secrets.

Product Overview

- Ingest: Upload emails (EML), documents (PDF/TXT), images (OCR), and containers (PST placeholder). Each email is split to its own record and linked into threads.
- Search: Ask in plain English; the system uses BM25 + vectors (hybrid) to find relevant items, with optional structured filters.
- Organise: Tags, claims (tree), link evidence to claims, and program events for chronology.
- Rebut: Provide opponent text; receive AI-drafted rebuttals with inline citations like [EV#123].
- Export: Generate Word (claims/rebuttal) and CSV (chronology) reports.

High-Level Architecture

```
graph LR
    User[User] --> FE[Frontend (React + TS)]
    FE --> API[Backend API (FastAPI)]
    subgraph Backend
        API --> DB[(PostgreSQL)]
        API --> OS[OpenSearch]
        API --> AI[OpenAI Embeddings + GPT]
        API --> FS[(Evidence Files)]
        API --> X[Exports (DOCX/CSV)]
    end
    note over API: All AI and secrets server-side
```

Technology Stack

- Frontend: React + TypeScript (Vite), desktop-first UI, AI Command Bar, Evidence Table, Preview Drawer, Claims Tree, Rebuttal editor, Chronology view.
- Backend: FastAPI + Pydantic; SQLAlchemy ORM; Alembic migrations; uvicorn.
- Database: PostgreSQL for cases, evidence, email metadata, tags, claims, rebuttal segments, programme events.
- Search: OpenSearch index `vericase_evidence` with `knn_vector` (1536 dims) + BM25; FAISS fallback supported.
- AI: OpenAI embeddings (default `text-embedding-3-small`) and GPT (`gpt-4o-mini`) for drafting; deterministic dummy fallback when keys are absent.
- OCR/PDF: PyMuPDF for text; Tesseract OCR fallback for image-only pages; PIL for image handling.

- Email: Robust parsing and threading from `.eml` ; `.pst` acknowledged as future work (stored only for now).
- Exports: `python-docx` for Word; CSV for chronology.

Core Components

- Frontend App
 - Pages: Cases, Evidence, Claims, Rebuttal, Chronology, Exports.
 - AI Command Bar: Modes `Filter` and `Evidence` (clarify then search).
 - Talks to API only; no direct service credentials in the browser.
- API Surface (FastAPI)
 - Routers mounted under `/api : health , cases , evidence , search , claims , rebuttal , chronology , exports , tags , programme , ai-command` .
 - Notable endpoints: evidence upload, hybrid search, AI query analysis/refinement, rebuttal generation, and document exports.
- Ingestion Pipeline
 - Detects file type; extracts text/metadata.
 - `.pdf` → PyMuPDF text; OCR fallback via Tesseract when needed.
 - `.eml` → parsed headers, threading (`thread_token` , `in_reply_to`), attachments captured.
 - Images → OCR path when available.
 - `.pst` → placeholder (“stored only”) with user-visible warning.
- Persistence
 - Evidence items store text, parties, date, thread, attachments; email metadata linked 1:1 where applicable.
 - Case-scoped tags, claim trees with evidence links, rebuttal segments, and programme events.
- Search & Indexing
 - Each evidence record is indexed to OpenSearch with an embedding vector (1536) and metadata (case, type, parties, date, tags).
 - Hybrid retrieval merges BM25 and vector scores with tunable `VECTOR_ALPHA` , `VECTOR_BM25_TOPN` , `VECTOR_HYBRID_TOPK` .
 - `VECTOR_BACKEND` may be `opensearch` or `faiss` (local fallback).
- AI Workflows
 - Command Bar `/ai/analyze-query` extracts filters or returns clarifying questions; `/ai/refine-query` converts Q&A to search params.
 - Rebuttal segments accept opponent text; server drafts rebuttals with cited evidence.

Data Model (MVP)

- Case: `id` , `name` , `description` , `owner_email` .
- EvidenceItem: `id` , `case_id` , `type` , `title` , `text_content` , `summary` , `date` , `party_from` , `party_to` , `thread_id` , `has_attachment` , `checksum` , `source_path` .
- EmailMessage: `evidence_id` , `thread_token` , `message_id` , `in_reply_to` , `subject` , `sender` , `recipients` , `cc` , `bcc` , `date_sent` , `headers_raw` , `body_html` .
- Attachment: `evidence_id` , `filename` , `content_type` , `file_size` , `storage_path` .
- Tag + EvidenceTag: case-scoped labels linked to evidence.
- Claim + ClaimEvidence: tree of claims with evidence links and optional relevance.
- RebuttalSegment: opponent text and drafted rebuttal for a case.
- ProgrammeEvent: dated events for chronology.

Hybrid Search Details

- Index: `vericase_evidence` with `knn_vector` embedding field (HNSW/cosine via nmslib), plus BM25 text fields.
- Query Flow
 - Optional: embed query text (OpenAI or dummy) to 1536-dim vector.
 - Fetch BM25 hits (size= `VECTOR_BM25_TOPN`), vector hits (size= `VECTOR_HYBRID_TOPK`), then weighted merge by `VECTOR_ALPHA` .
 - Filters: `type` , `party_from` , `party_to` , `tags` , date range.
- Fallback: FAISS local store supports upsert/query when `VECTOR_BACKEND=faiss` .

Files & Storage

- Dev storage path: `backend/var/evidence/` for uploaded artifacts.
- OpenSearch index creation helper: `python -m app.search.create_index` .
- All index writes refresh in dev for immediate searchability.

Security & Privacy

- No API keys in the frontend; all AI calls are server-side.
- Default dev CORS enabled; production hardening to be applied at deployment.
- PII handling: keep raw files local in dev; swap to S3/Blob in prod with identical API contract.

Deployment

- Local Dev: Docker for Postgres + OpenSearch; `uvicorn` for API; Vite dev server for frontend.
- Config via environment variables:
 - `OPENAI_API_KEY` , `OPENAI_MODEL_GPT`
 - `DATABASE_URL`
 - `OPENSEARCH_URL` , `OPENSEARCH_USER` , `OPENSEARCH_PASS`
 - `EMBEDDINGS_PROVIDER` (default `dummy`), `EMBEDDINGS_MODEL` (default `text-embedding-3-small`)
 - `VECTOR_BACKEND` (`opensearch` | `faiss`), `VECTOR_ALPHA` , `VECTOR_BM25_TOPN` , `VECTOR_HYBRID_TOPK`
 - `ENV` (`dev` | `prod` | `test`)

Non-Functional

- Performance: Evidence table targets ≥ 100 items with responsive preview.
- Reliability: Ingestion records warnings instead of hard-failing; deterministic embedding fallback without keys.
- Observability: Simple app logs; extend with structured logging and request IDs in prod.

Appendix: Representative API Endpoints

- Health: `GET /api/health`
- Cases: `POST /api/cases` , `GET /api/cases` , `GET /api/cases/{id}`
- Evidence: `POST /api/cases/{id}/upload` , `GET /api/cases/{id}/evidence` , `GET /api/evidence/{evidence_id}`
- Search: `GET /api/cases/{id}/search` (supports filters + optional embedding)

- AI Command: POST /api/ai/analyze-query , POST /api/ai/refine-query , POST /api/ai/suggest-filters
- Claims: POST /api/claims , GET /api/cases/{id}/claims , POST /api/claims/{claim_id}/evidence
- Rebuttal: POST /api/rebuttal/segments , POST /api/rebuttal/segments/{segment_id}/generate
- Chronology: GET /api/cases/{id}/chronology
- Exports: GET /api/cases/{id}/export/claims.docx , .../rebuttal.docx , .../chronology.csv
- Tags: POST /api/tags , GET /api/cases/{id}/tags
- Programme: POST /api/programme/events , GET /api/cases/{id}/programme/events

This rewrite aligns with docs/TECH_SPEC.md and the current backend/frontend implementation. If any detail here conflicts with the code, treat the code as canonical and please flag inconsistencies for correction.